Social Media Content Recommendation Algorithms and the Dissemination of Misinformation

by Serafina Smith

Abstract

The dissemination of misinformation on social media platforms is a critical issue, and it is largely driven by content recommendation algorithms designed to maximize user engagement. These algorithms rely on popularity bias to recommend content, which often increases the spread of misinformation. This project builds on previous research by selecting successful strategies from past misinformation classification models to create new models, comparing the performance of the models, and integrating the highest performing model into a mock recommendation algorithm. The results show that this integration can effectively filter out misinformation, with the best classification model achieving an accuracy rate of approximately 87%. The findings prove that, with the help of misinformation classification models, social media companies can improve their content recommendation algorithms to mitigate the spread of harmful and false information, creating a more reliable online environment.

Introduction

Popular social media platforms propel the spread of misinformation, in large part due to the algorithms that are implemented to recommend content to users. As explained by Fernandez et al. (2021), social media companies design these recommendation algorithms with the goal of extending users' engagement and time spent on the social media platform, in order to maximize their exposure to advertisements displayed on their social media feeds. Recommendation algorithms achieve this goal by promoting content that users are likely to consume. One approach to recommending engaging content is through promoting information that is trending–getting more clicks or shares–on the platform. This phenomenon, which many recommendation algorithms suffer from, is known as popularity bias. Popularity bias has been shown to increase the circulation of misinformation on social media platforms (Fernandez et al., 2021). The consumption of misinformation is incredibly dangerous for society at large, as it lures individuals into an alternative version of reality that can lead them to act in a way that is harmful for the real world, such as voting against policies that support modern science and medicine. For this reason, it is necessary to investigate the relationship between recommendation algorithms and the spread of misinformation, and explore how these algorithms can be adapted to mitigate this issue effectively.

Literature Review

Fernandez et al. (2021) analyze how some of the most widely used recommendation algorithms contribute to the spread of misinformation on Twitter and, based on this analysis and a review of the existing research, provide a set of guidelines to modify these algorithms. They conclude that the primary way to reduce the misinformation amplification of recommendation algorithms is to target the issue of popularity bias. Multiple adaptation methods addressing popularity bias are proposed, including pre-recommendation, within the model, and post-recommendation adaptations. One possible solution is to target the content itself, re-ranking the items based on the "reliability." They suggest that "content that ranks low on credibility could be either discarded (pre-adaptation) or re-ranked (post-adaptation)" (p. 26).

One way to rank the reliability of content is by building a misinformation classification model, which ranks reliability in a binary way, as either misinformation or true misinformation. Prior research has explored various strategies to build the optimal model that can detect and classify misinformation.

One of those strategies is creating and testing machine learning methods against each other as well as in combination with one and other. Neeraj et al. (2023) tested several machine learning models, including K-Nearest Neighbors (KNN), Decision Trees, Random Forest, Gradient Boosting, and custom ensemble models like Stacking and Maximum Voting Classifier. Their custom ensemble, combining KNN, Support Vector Classifier, and Logistic Regression (LR), achieved 91.5% accuracy in classifying news as true or fake. Barbar (2023) compared Naive Bayes, Support Vector Machines (SVM), Logistic Regression, and Decision Trees, alongside two vectorization techniques, Countvectorizer and TF-IDF. The results found that SVM combined with the TF-IDF vectorizer achieved the highest accuracy in identifying fake news.

Deep learning based models have also been tested for fake new detection, often incorporating convolutional neural networks (CNN) and/or long short-term memory (LSTM). For instance, Ivancová et al. (2021) detected fake news in Slovak-language articles using CNN and LSTM neural networks. While the models showed comparable performance, the LSTM model was slightly more effective, achieving higher accuracy and better recall for both categories. Buzea et al. (2022) evaluated RNNs with LSTM and gated recurrent units, CNNs, and a pre-trained Romanian BERT model, RoBERTa, alongside traditional classifiers like Naïve Bayes and SVM. The results indicated that CNN outperformed both the traditional classifiers and BERT-based models, making it the most effective approach. Abdulrahman and Baykara (2020) used four text feature extraction methods (TF-IDF, count vector, character-level vector, N-gram) and applied them to a fake news dataset, comparing 10 different ML and DL classifiers. CNN performed the best, with accuracies between 81-100%.

Some researchers have also attempted to combine machine learning and deep learning to create more comprehensive models. Maham (2024) had promising results by combining the machine learning models Random Forest and SVM and the deep learning models BERT, RoBERTa, and GRU in two phases: first without adversarial training, then with it, using the Fast Gradient Sign Method (FGSM) to enhance robustness.

This project applies some of these strategies that have been proven to be successful in prior research, including KNN, SVM, LR, and CNN + LSTM, to develop various misinformation classification models tailored to the content of social media posts. While previous studies have focused on the theoretical component to improving recommendation algorithms and addressing popularity bias, there is a lack of work that explicitly integrates misinformation classification into the recommendation process itself. For this reason, this project responds to Fernandez et al.'s (2021) suggestions, integrating the highest performing classification model into a mock recommendation system that mimics Twitter's algorithm, so that low reliability content is discarded. In doing so, it aims to reduce the amplification of misinformation and contribute to the development of more reliable social media recommendation algorithms.

Materials and Methods

Datasets

A combination of four pre-existing, publicly available datasets were used to train and test the classification models. The datasets were chosen for their size, topic diversity, and content type. A summary of the following information can be found in Table 1. The datasets were selected to closely match the content commonly found on social media platforms, to ensure relevance to real-world misinformation detection.

The first dataset, TruthSeeker2023, consists of over 180,000 labeled tweets (Dadkhah et al., 2024). These tweets are related to 700 real and 700 fake news stories from the Politifact Dataset, which contains statements fact-checked by experts from Politifact.com. Keywords were manually created for each news story topic, and tweets were extracted using the Twitter API. To label the tweets, crowdsourcing through Amazon Mechanical Turk was used, with a majority agreement algorithm determining whether an item matched a real or fake news story. This dataset covers a diverse range of topics and is one of the largest for fake news detection on Twitter. Along with tweet content, it includes user metadata. Content in the TruthSeeker2023 dataset was given the label "TRUE" or "FALSE," thereby not specifically addressing nuanced statements that may be in between true and false information; those undetermined tweets were left out of the dataset. This dataset primarily focuses on classifying content as either true or false based on verified facts, aligning with a binary approach to misinformation detection.

The second dataset is made up of 10,700 manually annotated social media posts and articles of real and fake news on COVID-19 (Das et al., 2021). Real news includes tweets from verified sources like the WHO and the CDC, defined as providing useful COVID-19 information. Fake news consists of verified false claims about COVID-19 collected from fact-checking websites like PolitiFact and Snopes. The dataset features include unique words, average words per post, and average characters per post for both real and fake news. Similar to the TruthSeeker2023 dataset, content in the COVID-19 dataset was given the label "real" or "fake," forgoing the inclusion of nuanced statements in the dataset.

The third dataset contains 5,266 tweets relating to Monkeypox that were published in July 2022 (Crone, 2022). Labels were determined based on trusted public health information sources, including the WHO, the CDC, and the UK National Health Service, alongside fact-checking organizations like Politifact and Snopes. Content in the Monkeypox dataset was given labels according to two different labeling systems: 0 for "misinformation" and 1 for "other" or 0 for "misinformation," 1 for "other," and 9 for "good information." The ternary approach allows for capturing statements that may not fit into the binary categories of true or false information, which is often the case in real-world situations.

The fourth dataset, known as LIAR, contains 12,800 manually labeled short statements from Politifact.com (Wang, 2017). These statements, collected over a decade, cover a wide range of topics and were evaluated for truthfulness by Politifact editors. Content in the LIAR dataset was given the label of "TRUE," "mostly-true," "half-true," "barely-true," "FALSE," or "pants-fire." This labeling system captures more nuanced information than the other three datasets, using a spectrum from true to false, which better reflects the range of information encountered on social media.

Methodology

To build a successful misinformation classification model, the plan was to compare successful methods from prior research. First, three machine learning models (KNN, SVM, and LR) were built individually and then combined into a stacked meta-model. Next a deep learning model was created, combining CNN and LSTM. Finally the stacked machine learning model was combined with the deep learning model to create a final model. An overview of the methodology is represented in Figure 1.

Before model development, the data preparation consisted of loading and pre-processing each dataset individually. For this project, the datasets were simplified to only include two columns, one with the text of the post/article/statement and one with the label of that content. One-hot encoding was performed to assign the value of 0 to “TRUE,” “real,” or “good information” and 1 to “FALSE,” “fake,” or “misinformation.” For the LIAR dataset, 0 was assigned to content labeled as “TRUE,” “mostly-true,” and “half-true,” and 1 was assigned to content labeled as “barely-true,” “FALSE,” and “pants-fire.” Stratified sampling was also used on each dataset, to ensure 50% of each dataset was true information and 50% was misinformation. The TruthSeeker2023 dataset was downsampled, so that the final combined dataset was made up of 40,212 entries from TruthSeeker2023 (~70%), 6,120 entries from COVID-19 (~10.7%), 2,138 entries from Monkeypox (~3.7%), and 8,976 entries from LIAR (~15.6%), as represented in Figure 2. The combined dataset was shuffled, and ended with 28,896 rows labeled as true information and 28,550 rows labeled as misinformation, adding to 57,446 total rows. After creating the combined dataset, it was cleaned to check for NaN values, to convert each entry to lowercase, and to remove mentions, hashtags, URLs, special characters, and emojis.

Next, feature engineering was implemented to vectorize the dataset using TfidfVectorizer (TF-IDF), with max features equal to 5,000. TF-IDF, which stands for Term Frequency-Inverse Document Frequency, is a foundational technique in natural language processing (NLP), known for its computational efficiency and effectiveness in representing textual data (Maham, 2024). This method assigns weights to words based on their significance within a document relative to the entire dataset. Prior research has shown that TF-IDF can effectively highlight key terms frequently associated with misinformation/fake news, such as "election fraud" and "propaganda," which appear more prominently in fake news articles compared to legitimate sources (Maham, 2024).

The dataset was randomly split, so that 80% was designated for training and 20% for testing. A model was created that used K-Nearest Neighbors, a non-parametric classifier that predicts the category of a data point based on its proximity to neighboring points (Neeraj, 2023). Using TF-IDF vectors, KNN calculates distances to identify the nearest neighbors, assigning the data point to the category most common among its closest neighbors in the feature space (Neeraj, 2023). Another model was created to use Support Vector Machine, which constructs optimal decision boundaries using hyperplanes in a high-dimensional feature space, achieving high accuracy even in nonlinear, high-dimensional, and complex scenarios (Deris, 2011). The last machine learning model was created using Logistic Regression. This algorithm uses the sigmoid function to estimate the probability of an occurrence. If the resulting value falls below a specified threshold, the category is classified as false; otherwise, it is classified as true (Neeraj, 2023).

The next model that was created uses a stacking ensemble approach, combining the prior three machine learning models. In this model, the KNN, SVM, and LR models are defined as base estimators, which independently learn patterns in the training data. A Logistic Regression model is then used as the meta-model, or the final estimator, taking the predictions (or outputs) of the base models as its input to make the final predictions.

The next step was the creation of a deep learning model, using CNN and LSTM. CNNs extract local patterns from text using convolutional filters, apply ReLU for non-linearity, and use pooling layers to reduce dimensionality (Choubey, 2020). LSTMs are neural networks designed to capture long-term dependencies in sequential data by using an architecture with gates and a cell state (Padalko et al., 2024). These work together to selectively retain or discard information, making LSTMs highly effective for tasks like time-series data classification and prediction (Padalko et al., 2024). Instead of using TF-IDF, tokenization using Tokenizer was used for feature encoding, with the number of words set to 5,000. Tokenization involves breaking down texts into smaller units, or tokens, which are then converted into vectors that the machine can interpret (Abdulrahman and Baykara, 2020). Tokens can be individual characters, subwords, or whole words, and tokenization can be classified into three types: word tokenization, character tokenization, and subword tokenization (Abdulrahman and Baykara, 2020). The deep learning model starts with an embedding layer that converts tokenized text into dense vectors. It then applies two 1D convolutional layers with max-pooling to capture local patterns in the text. The CNN layers are followed by an LSTM layer to capture sequential dependencies in the text. Dropout is applied throughout the model to reduce overfitting. Finally, the model includes dense layers for classification, with the output layer using a sigmoid activation function for binary classification. The model is trained using the Adam optimizer and binary cross-entropy loss.

Finally, the stacked machine learning model and the deep learning model were combined, by first obtaining predictions from both models. The stacking classifier output binary predictions and the CNN + LSTM model output probabilities for each test instance. These predictions were then stacked into a new feature set, which was used as an input for a final meta-model, which was a Logistic Regression classifier. This meta-model was trained on the combined outputs.

Parameter tuning was performed on the individual machine learning models–KNN, SVM, and LR–along with the deep learning model. The optimal hyperparameters were found using GridSearch for the ML models and KerasTuner for the DL model. For KNN, the best parameters were {'metric': 'euclidean', 'n_neighbors': 12, 'weights': 'distance'}. For SVM, the best parameters were {'C': 0.1, 'tol': 0.001, 'max_iter': 1000}. For LR, they were {'penalty': 'l2', 'C': 1, 'solver': 'liblinear', 'max_iter': 100}. Finally, for the deep learning model, they were {'embedding_dim': 250, 'filters_1': 32, 'kernel_size_1': 5, 'filters_2': 64, 'kernel_size_2': 3, 'dropout_cnn': 0.4, 'lstm_units': 96, 'dropout_lstm': 0.2, 'dense_units': 128, 'dropout_dense': 0.3, 'learning_rate': 0.0005} with a tuning process of 4 epochs and an initial trial configuration.

Lastly, a mock recommendation algorithm was created to mirror the basic structure of Twitter's content recommendation algorithm, as outlined in an article posted by Twitter (2023). This mock model's key components include candidate fetching within-network and out-of-network, ranking content based on relevance scores (based on metrics like likes, retweets, and replies), and heuristics-based filtering (like author diversity and blocking users or keywords) (Twitter, 2023).

This standard model was adapted to include a preprocessing step, which filtered out misinformation using the stacked machine learning model. The remaining content would then be passed through the typical recommendation system to generate a curated list of reliable recommended posts.

Results

Standard metrics were used to evaluate the performance of each classification model, including accuracy, precision, recall, F1 score, and time. The performance of each model was slightly different each time the combined dataset was randomly split into 80% training and 20% testing, but the results were fairly consistent relative to one and other.

After data cleaning and hyperparameter tuning, the KNN model achieved an accuracy of 82.35%. For class 0, it recorded a precision of 0.93, recall of 0.71, and an F1 score of 0.80. For class 1, the precision was 0.76, recall was 0.94, and the F1 score was 0.84. The SVM model achieved an accuracy of 85.03%. For class 0, it had a precision of 0.84, recall of 0.87, and an F1 score of 0.86. For class 1, the precision was 0.86, recall was 0.83, and the F1 score was 0.84. The Logistic Regression model achieved an accuracy of 84.92%. For class 0, it recorded a precision of 0.84, recall of 0.87, and an F1 score of 0.85. For class 1, the precision was 0.86, recall was 0.83, and the F1 score was 0.84. The stacked ensemble model (combining KNN, SVM, and LR) achieved an accuracy of 86.75%. For class 0, it had a precision of 0.87, recall of 0.86, and an F1 score of 0.87. For class 1, the precision was 0.86, recall was 0.87, and the F1 score was 0.87. The deep learning model (CNN + LSTM) achieved an accuracy of 85.47%. For class 0, it recorded a precision of 0.87, recall of 0.84, and an F1 score of 0.85. For class 1, the precision was 0.84, recall was 0.87, and the F1 score was 0.86. The final stacked model, which combined the outputs of the stacking ensemble and the deep learning model, achieved an accuracy of 86.47%. For class 0, it had a precision of 0.87, recall of 0.87, and an F1 score of 0.87. For class 1, the precision was 0.86, recall was 0.86, and the F1 score was 0.86. This information is represented in Figure 3. Figure 4 represents the same metrics for the models before data cleaning and hyperparameter tuning were implemented. Precision, recall, and F1 scores were graphed using the average value of class 0 and class 1.

The time required for training and making predictions varied across the models. The KNN model took 0.4273 seconds to fit and 377.1730 seconds to make predictions. The SVM model required 1.7048 seconds to fit and 0.1975 seconds for predictions. The LR model took 1.5218 seconds to fit and 0.3267 seconds to make predictions. The stacked model (KNN, SVM, LR) required 632.3453 seconds for fitting and 380.9210 seconds for predictions. The deep learning model took 287.8457 seconds for training and 6.3976 seconds for predictions. The final stacked model required 0.0135 seconds to train and 0.0013 seconds to make predictions.

Discussion

Data cleaning and hyperparameter tuning caused an increase in performance across all models, with an average improvement in accuracy of 4.96 percentage points. After tuning, the results show that the individual KNN model performed the worst, and was the second slowest, with an accuracy of only 82.35%. The model has high precision (0.93) for class 0 (true information), meaning that when the model predicts information as true, it is correct 93% of the time, with a low false positive rate (misclassifying misinformation as true). However, its recall for class 0 is

lower at 0.71, meaning that 29% of the actual true information in the dataset is misclassified as misinformation (false negatives). For class 1 (misinformation), the model has a low precision of 0.76, meaning 76% of the predictions labeled as misinformation are accurate and 24% are false positives (true information incorrectly labeled as misinformation). Its recall for misinformation is very high (0.94), correctly identifying 94% of the actual misinformation in the dataset, though this comes at the cost of misclassifying a significant portion of true information as misinformation. The model likely struggled due to the high-dimensional, sparse matrix created by TF-IDF, where most entries were zero. Calculating distances between TF-IDF vectors in such a sparse, high-dimensional space reduces the effectiveness of distance metrics, as is known as the "curse of dimensionality" (Halder, 2024). Therefore, the model would likely perform better with a dataset that has fewer features and lower dimensionality.

The remaining models—SVM, LR, stacked ML, CNN + LSTM, and stacked ML + DL—performed similarly, with accuracies between 85% and 87% and consistently high precision, recall, and F1 scores. The stacked ML model was slightly the best performer, though its performance was comparable to the stacked ML + DL model. It was surprising that the CNN + LSTM model did not outperform the machine learning models, as those methods are designed to capture complex patterns. This may be because, although the dataset was too high dimensional for KNN, it was not large or complex enough for deep learning to fully exploit its potential (Choubey, 2020; Padalko et al., 2024). Instead, the stacked ML model slightly outperformed others, with an accuracy of 86.75%, by combining the complementary strengths of individual ML models. This means that the mock Twitter recommendation algorithm that was adapted to incorporate this model is able to correctly identify and filter out misinformation 86.75% of the time.

These results align with prior research on misinformation classification, where model accuracies range from 81% to 100%, as mentioned above. A key takeaway from this project is the importance of training data in building accurate misinformation classification models for integration into social media recommendation algorithms. Ideally, the data should reflect the types of posts on the platform, covering a broad range of topics to ensure accurate identification across diverse content. However, creating large, high-quality, and well-labeled datasets requires substantial time and resources. That being said, social media companies have the ability to further improve the performance of the mock recommendation algorithm, given their access to substantial platform-specific data and resources.

One limitation of this project has to do with the binary nature of the misinformation classification models. As mentioned in above sections, the real world information may not fit into binary categories, instead verging on the truth or a lie. Binary classification models may not be well suited for times like these. Additionally, misinformation topics evolve rapidly, requiring models to be continuously updated with new data to effectively identify misinformation on new, hot-button issues. The models created in this project also did not incorporate metadata, such as users, number of likes, tags, or date of post, into the decision process, instead focusing solely on post content. Lastly, when attempts were made to test the models on larger quantities of data, Google CoLab ran out of RAM and was unable to fulfill the requests, thereby limiting the dataset used to contain 57,446 entries, despite access to more data.

With these limitations in mind, future research could implement changes to improve upon the robustness of the models in this project by incorporating a more nuanced approach to content reliability, ranking it on a spectrum of trustworthiness rather than classifying it as true or false. Additionally, future models could incorporate changes to rank the reliability of the user posting the content, using tools that allow for the detection of malicious actors (bots, sockpupets), as suggested by Fernandez et al. (2021). Finally, with access to cloud computing services, larger datasets could be used for model training and testing to improve the overall effectiveness of the model.

Conclusion

In conclusion, this project offers a viable approach to mitigating the dissemination of misinformation on social media platforms. This was accomplished by using prior research to build accurate misinformation classification models, using individual machine learning models, a stacked machine learning model, a deep learning model, and a combined machine learning and deep learning model. The most successful model–the stacked machined learning model, which achieved an accuracy rate of approximately 87%–was then integrated into a mock recommendation algorithm, based on the architecture of Twitter's recommendation pipeline, proving that misinformation can be successfully filtered out of users' feeds. This project also highlights the challenges in obtaining high-quality, platform-specific data and suggests that further improvements can be achieved with more diverse and expansive datasets as well as incorporating a more nuanced approach with metadata and a spectrum of classification instead of a binary model. Overall, this work contributes to the ongoing effort to create more reliable and responsible social media content recommendation systems that can reduce the harmful effects of misinformation on public knowledge and democracy at large.

References

Abdulrahman, A., & Baykara, M. (2020). Fake news detection using machine learning and deep learning algorithms. In *Proceedings of the 3rd International Conference on Advanced Science and Engineering (ICOASE)* (pp. 18–23). IEEE. https://doi.org/10.1109/ICOASE51841.2020.9436605

Babar, M., Ahmad, A., Tariq, U. M., & Kaleem, S. (2023). Real-time fake news detection using big data analytics and deep neural network. *IEEE Transactions on Computational Social Systems*. https://doi.org/10.1109/TCSS.2023.3309704

Buzea, M. C., Trausan-Matu, S., & Rebedea, T. (2022). Automatic Fake News Detection for Romanian Online News. *Information, 13*(3), 151. https://doi.org/10.3390/info13030151

Choubey, V. (2020, July 22). *Text classification using CNN*. Medium. https://medium.com/voice-tech-podcast/text-classification-using-cnn-9ade8155dfb9

Crone, S. (2022). Monkeypox misinformation: Twitter dataset. *Kaggle*. https://doi.org/10.34740/KAGGLE/DS/2352848

Dadkhah, S., Zhang, X., Weismann, A. G., Firouzi, A., & Ghorbani, A. A. (2024). The largest social media ground-truth dataset for real/fake content: TruthSeeker. *IEEE Transactions on Computational Social Systems, 11*(3), 3376–3390. https://doi.org/10.1109/TCSS.2023.3322303

Das, S. D., Basak, A., & Dutta, S. (2021). A heuristic-driven ensemble framework for COVID-19 fake news detection. *arXiv preprint*. https://arxiv.org/abs/2101.03545

Deris, A. M., Zain, A. M., & Sallehuddin, R. (2011). Overview of Support Vector Machine in modeling machining performances. *Procedia Engineering, 24*, 308–312. https://doi.org/10.1016/j.proeng.2011.11.2647

Fernández, M., Bellogín, A, & Cantador, I. (2021). Analysing the effect of recommendation algorithms on the amplification of misinformation. arXiv:2103.14748. Retrieved from https://arxiv.org/abs/2103.14748

Halder, R. K., Uddin, M. N., Uddin, M. A., et al. (2024). Enhancing K-nearest neighbor algorithm: A comprehensive review and performance analysis of modifications. *Journal of Big Data, 11*, 113. https://doi.org/10.1186/s40537-024-00973-y

Ivancová, K., Sarnovský, M., & Maslej-Krcšňáková, V. (2021). Fake news detection in Slovak language using deep learning techniques. In *2021 IEEE 19th World Symposium on Applied Machine Intelligence and Informatics (SAMI)* (pp. 255–260). IEEE. https://doi.org/10.1109/SAMI50585.2021.9378650

Maham, S., Tariq, A., Khan, M. U. G., et al. (2024). ANN: Adversarial news net for robust fake news classification. *Scientific Reports, 14*, 7897. https://doi.org/10.1038/s41598-024-56567-4

Neeraj, S., Singh, L., Tripathi, S., & Malik, N. (2023). Detection of fake news using machine learning. In *Proceedings of the 13th International Conference on Cloud Computing, Data Science & Engineering (Confluence)* (pp. 20-24). IEEE. https://doi.org/10.1109/Confluence56041.2023.10048819

Shahzad, K., Khan, S. A., Iqbal, A., Shabbir, O., & Latif, M. (2023). Determinants of fake news diffusion on social media: A systematic literature review. *Global Knowledge, Memory and Communication*. https://doi.org/10.1108/GKMC-06-2023-0189

Twitter. (2023). Twitter's Recommendation Algorithm. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm

Wang, W. Y. (2017). Liar, liar pants on fire: A new benchmark dataset for fake news detection. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL 2017) (short paper). Vancouver, BC, Canada. July 30–August 4. ACL.

Appendix

Link to Project Code:
https://colab.research.google.com/drive/1j7nM5MZw7C4Ap23btPsu6L-y015Cb8vY?usp=sharing

Dataset	Content Type	Size	Topic
TruthSeeker2023	Tweets	180,000	Diverse Range
COVID-19 Fake News Detection	Social Media Posts and Articles	10,700	COVID-19
Monkeypox Misinformation	Tweets	5,266	Monkeypox
LIAR	Statements from Politifact.com	12,800	Diverse Range

Table 1. Summary of Datasets.

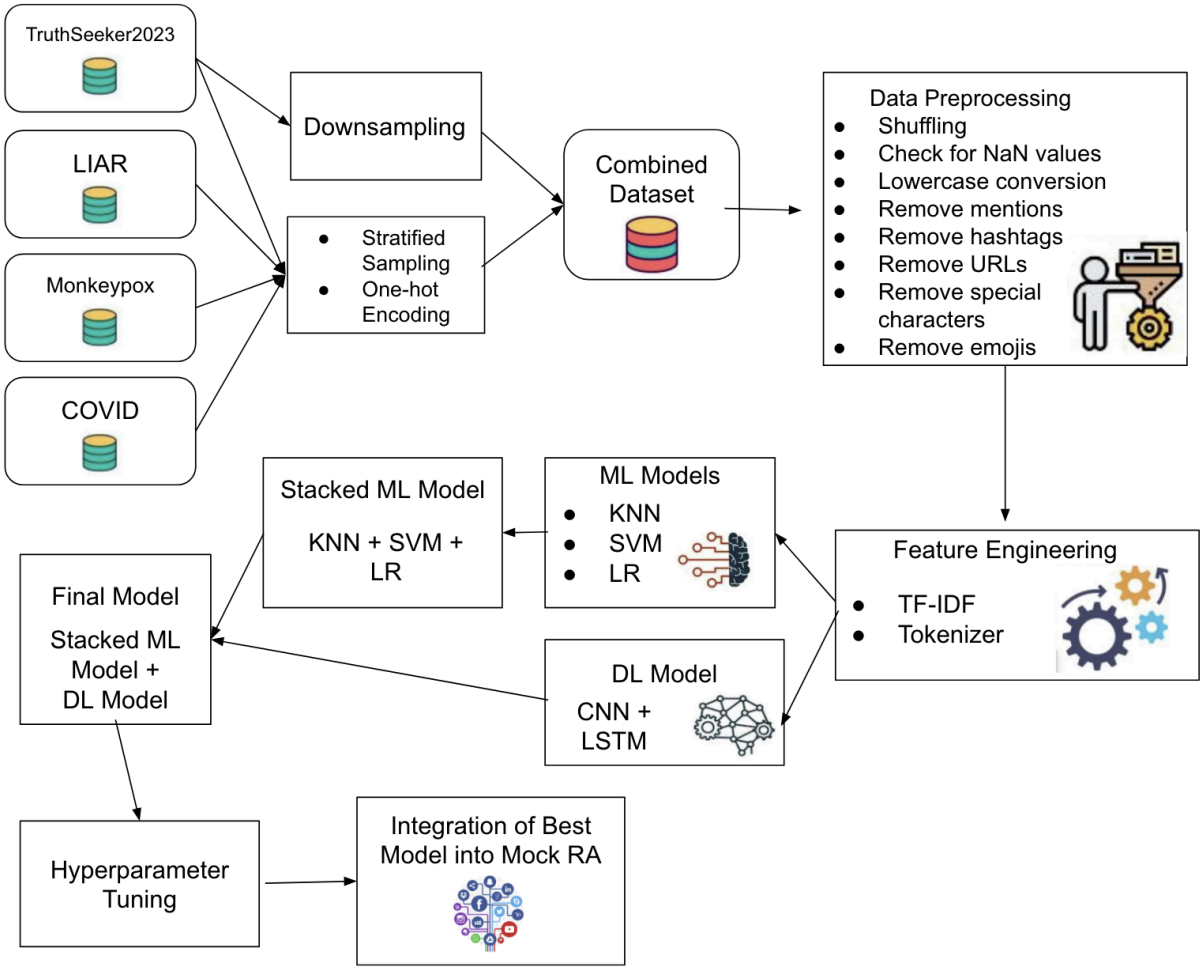

Figure 1. Flow Diagram of Methodology; Images from Maham et al. (2024)

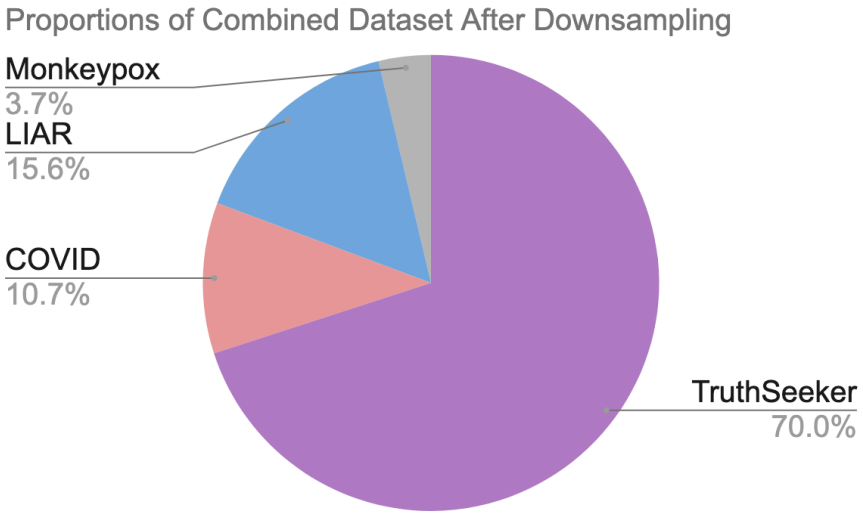

Figure 2. Pie Chart Representing Proportions of Combined Dataset After Downsampling

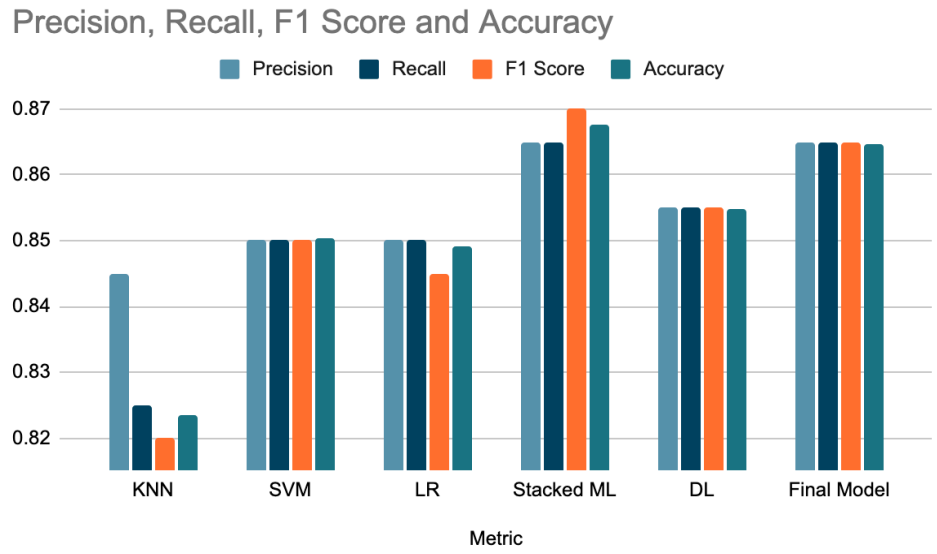

Figure 3. Model Comparison After Data Cleaning and Hyperparameter Tuning

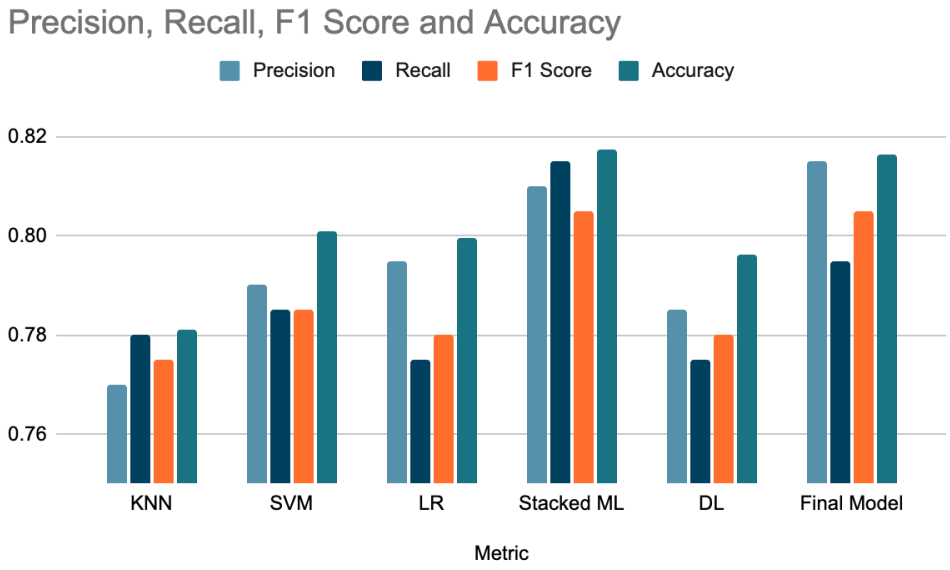

Figure 4. Model Comparison Before Data Cleaning and Hyperparameter Tuning